

Vulnerability Triage Report

Generated by vulntrriage · 2026-07-01 21:23

Executive Summary

This report covers 20 vulnerability finding(s) triaged by the LLM-driven pipeline. Of these, 12 are rated High exploitability, 3 Medium, and 5 Low. The highest-priority finding is: Open Redis 3.2.0 without authentication, internet-facing (host 192.168.1.10, risk score 0.99). Findings are ranked by a composite risk score combining CVSS, LLM-assessed exploitability, and asset criticality. Recommended remediation steps follow each finding.



Risk Breakdown

#1 Open Redis 3.2.0 without authentication, interne...	0.99
#2 Apache Log4j 2.14.1 detected in a Java web app, ...	0.9
#3 Apache Struts 2.5.16 running a legacy action, in...	0.9
#4 Jenkins 2.426 running with CLI exposed, internet...	0.89
#5 Atlassian Confluence 7.4.0 server exposed to the...	0.89
#6 Elasticsearch 1.4.2 without authentication and s...	0.89
#7 Apache HTTP Server 2.4.49 with path traversal, i...	0.89
#8 Docker Engine API exposed without TLS on port 23...	0.89
#9 PHP 8.0 running on Windows with CGI, internal-on...	0.89
#10 GitLab CE 16.1.0 self-hosted, account takeover p...	0.88

#11 Apache Tomcat 9.0.30 with manager app exposed
an...



#12 MongoDB 4.0 without authentication exposed on
th...



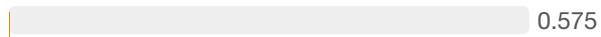
#13 MySQL 5.7.27 with anonymous account and weak
roo...



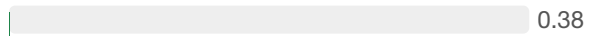
#14 Grafana 8.0.0 with arbitrary file read, internal...



#15 WordPress 5.4 with multiple outdated plugins, in...



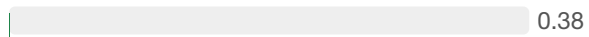
#16 PostgreSQL 15.3, patched, listening only on loca...



#17 Redis 7.2.3 with authentication enabled and ACL ...



#18 OpenSSH 9.6p1 fully patched on a bastion host, i...



#19 nginx 1.25.3, no known vulnerabilities, internet...



#20 OpenSSH 8.2p1 on internal host, patched against



...

Technical Findings

#1 High Open Redis 3.2.0 without authentication, internet-facing

Host: 192.168.1.10:6379

CVE: CVE-2022-0543

CVSS: 9.8

Risk score: 0.99 (asset criticality 1.0)

Context: The vulnerability involves Redis 3.2.0 being exposed without authentication on an internet-facing host, allowing remote attackers to execute arbitrary commands via the open CLI interface. This misconfiguration enables attackers to bypass access controls, read sensitive data, or modify the database contents, potentially leading to data exfiltration or system compromise. The business impact includes severe confidentiality and integrity breaches, as well as potential service disruption or lateral movement within the network.

Exploitability rationale: The combination of an unpatched Redis 3.2.0 version, complete lack of authentication, and public internet exposure creates a critical attack surface that allows remote attackers to easily execute arbitrary commands and achieve full system compromise.

Remediation: Remediate the critical remote code execution vulnerability by upgrading Redis to a patched version, enforcing authentication, and restricting network exposure to eliminate the attack surface.

1. Upgrade the Redis service on host 192.168.1.10 to version 5:6.0.16-1+deb11u2 or later to patch the Lua sandbox escape.
2. Configure redis.conf to require a strong password using the requirepass directive or implement ACL-based user authentication.
3. Bind the Redis server to localhost (127.0.0.1) or a specific internal interface only, ensuring port 6379 is not accessible from external networks.
4. Disable or restrict the EVAL and EVALSHA commands via rename-command directives if Redis scripting functionality is not required for the application.

RAG references: CVE-2022-0543

#2 High Apache Log4j 2.14.1 detected in a Java web app, internet-facing (CVE-2021-44228)

Host: 192.168.1.60:8080

CVE: CVE-2021-44228

CVSS: 10.0

Risk score: 0.9 (asset criticality 0.5)

Context: The Apache Log4j 2.14.1 vulnerability (CVE-2021-44228) affects the Java web application running on host 192.168.1.60, allowing attackers to execute arbitrary code via a crafted log message. Since the service is internet-facing, this flaw enables remote code execution from untrusted sources, potentially leading to full system compromise, data exfiltration, or denial of service. Given the CVSS score of 10.0, this critical vulnerability poses an immediate and severe threat to the organization's confidentiality, integrity, and availability.

Exploitability rationale: The internet-facing Apache Log4j 2.14.1 vulnerability (CVE-2021-44228) allows remote code execution with a CVSS score of 10.0, and public exploits are widely available, making it trivial for attackers to compromise the system.

Remediation: The internet-facing Java web application is vulnerable to remote code execution due to an outdated Log4j version (2.14.1) that lacks protection against JNDI lookups, requiring an immediate upgrade and interim hardening.

1. Identify and patch all instances of Log4j across the application stack, including transitive dependencies, by upgrading to version 2.17.1 or later (or 2.12.4 / 2.3.2 for legacy branches).
2. Configure the application to set 'log4j2.formatMsgNoLookups=true' or remove the JndiLookup class from the classpath as an interim mitigation measure.
3. Monitor application logs for JNDI lookup patterns and outbound LDAP/RMI connections to detect and respond to any exploitation attempts.

RAG references: CVE-2021-44228

#3 High Apache Struts 2.5.16 running a legacy action, internet-facing (CVE-2017-5638)

Host: 192.168.1.100:8080

CVE: CVE-2017-5638

CVSS: 10.0

Risk score: 0.9 (asset criticality 0.5)

Context: The Apache Struts 2.5.16 framework on host 192.168.1.100 is vulnerable to CVE-2017-5638, a Remote Code Execution flaw in the OGNL parser that allows attackers to execute arbitrary system commands. Since the service is internet-facing, an attacker can exploit this vulnerability remotely without authentication to gain full control over the server. This results in complete compromise of the web application and potentially the entire internal network, leading to severe business disruption and data loss.

Exploitability rationale: The finding is rated High because the vulnerability allows remote, unauthenticated RCE with a CVSS score of 10.0, and the service is publicly exposed on the internet.

Remediation: Remediate the critical remote code execution vulnerability in Apache Struts 2.5.16 by immediately upgrading to a patched version or applying strict input validation controls.

1. Immediately upgrade Apache Struts from version 2.5.16 to version 2.5.10.1 or higher, preferably to the latest stable 2.5.x or 6.x release.
2. If an immediate upgrade is not possible, apply vendor-specific mitigations and disable or remove the Struts REST plugin and any unused Struts features.
3. Configure the web server to reject or sanitize malformed Content-Type and Content-Disposition headers to prevent OGNL injection.
4. Monitor web server logs for exploitation attempts targeting the Struts framework and inspect for signs of compromise such as web shells.
5. Conduct a full audit of the server for unauthorized files and rotate all credentials that may have been exposed or compromised.

RAG references: CVE-2017-5638

#4 High Jenkins 2.426 running with CLI exposed, internet-facing (CVE-2024-23897)

Host: 192.168.1.70:8080

CVE: CVE-2024-23897

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: The vulnerability affects Jenkins 2.426 when the CLI is exposed to the internet, allowing remote attackers to execute arbitrary commands on the server by exploiting the 'Jenkins CLI' feature. An attacker can leverage this exposure to gain full control over the host system, potentially leading to data exfiltration, lateral movement, or denial of service. The business impact is severe, as it compromises the integrity and confidentiality of any data processed by Jenkins, including source code, build artifacts, and sensitive configuration details.

Exploitability rationale: The combination of an internet-facing exposure, a critical CVSS score of 9.8, and the ability to execute arbitrary commands allows for immediate and severe compromise with minimal attack complexity.

Remediation: Immediately restrict or disable the exposed Jenkins CLI endpoint to prevent unauthorized file access, then upgrade Jenkins to a patched version and rotate all compromised credentials.

1. Disable the Jenkins CLI over the HTTP endpoint or restrict access to authenticated administrators immediately.
2. Place the Jenkins instance behind a reverse proxy that enforces authentication and applies strict network access control lists (ACLs).
3. Upgrade Jenkins from version 2.426 to LTS 2.426.3 or a weekly release 2.442 or later to patch the vulnerability.
4. Rotate all credentials and secrets stored in Jenkins following the confirmation of the exposure.

RAG references: CVE-2024-23897

#5 High Atlassian Confluence 7.4.0 server exposed to the internet (CVE-2022-26134)

Host: 192.168.1.80:8090

CVE: CVE-2022-26134

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: The Atlassian Confluence server at 192.168.1.80 is vulnerable to CVE-2022-26134, a critical Remote Code Execution flaw in the JSP engine that allows attackers to execute arbitrary code via a crafted URL. An attacker exploiting this vulnerability could gain full control over the Confluence instance, potentially leading to data exfiltration, denial of service, or lateral movement within the network. Given the high CVSS score of 9.8, this exposure poses a severe risk to organizational confidentiality, integrity, and availability, especially since Confluence often hosts sensitive corporate documentation and knowledge bases.

Exploitability rationale: The combination of a critical Remote Code Execution vulnerability (CVSS 9.8) on a publicly exposed service allows attackers to easily achieve full control over the system without requiring additional authentication or complex conditions.

Remediation: Remediate the critical OGNL injection vulnerability by upgrading Atlassian Confluence to a patched version or applying the official workaround, followed by access log review and credential rotation.

1. Immediately upgrade Atlassian Confluence Server or Data Center to version 7.4.17 or a later fixed release (e.g., 7.13.7, 7.14.3, 7.15.2, 7.16.4, 7.17.4, 7.18.1, or 7.19.0).
2. If an immediate upgrade is not feasible, apply the official out-of-band patch or workaround provided by Atlassian.
3. Review server access logs for suspicious URI patterns that may indicate exploitation attempts.
4. Rotate all administrative credentials and investigate for any signs of persistence if exploitation is confirmed.

RAG references: CVE-2022-26134

#6 High Elasticsearch 1.4.2 without authentication and scripting enabled, internal (CVE-2015-1427)

Host: 192.168.1.90:9200

CVE: CVE-2015-1427

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: The Elasticsearch instance on 192.168.1.90 is running version 1.4.2 with both authentication and scripting disabled, creating a critical vulnerability (CVE-2015-1427) that allows remote attackers to execute arbitrary code via a crafted query. An attacker can exploit this by sending a malicious search request that triggers the scripting engine, leading to full server compromise, including the ability to read, modify, or delete data, and potentially pivot to other internal systems. The business impact is severe, ranging from complete data loss and unauthorized access to sensitive information to the potential for ransomware encryption and operational downtime.

Exploitability rationale: The combination of an unpatched version (1.4.2) with scripting enabled and no authentication, alongside a known critical remote code execution vulnerability (CVE-2015-1427) with a CVSS score of 9.8, allows for immediate full server compromise without complex attack conditions.

Remediation: Remediate the remote code execution vulnerability by upgrading Elasticsearch to a patched version, enforcing authentication and TLS, and disabling dynamic scripting to prevent sandbox escape.

1. Upgrade the Elasticsearch instance from version 1.4.2 to version 1.4.3 or later to apply the security patch for CVE-2015-1427.
2. Configure the Elasticsearch configuration file to disable dynamic scripting by setting 'script.disable_dynamic' to 'true'.
3. Enforce authentication and TLS encryption for all connections to the Elasticsearch service, ensuring port 9200 is not exposed to untrusted networks.
4. Restrict network access to the Elasticsearch service using a reverse proxy or firewall to allow connections only from known internal clients.
5. Plan a migration to a supported Elasticsearch version (5.x or later) where the scripting model is sandboxed differently to ensure long-term security.

RAG references: CVE-2015-1427

#7 High Apache HTTP Server 2.4.49 with path traversal, internet-facing (CVE-2021-41773)

Host: 192.168.1.140:80

CVE: CVE-2021-41773

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: Apache HTTP Server 2.4.49 is vulnerable to CVE-2021-41773, a critical path traversal flaw in the `mod_rewrite` module that allows attackers to read arbitrary files from the server's filesystem. An attacker exploiting this vulnerability can bypass web application security controls to access sensitive configuration files, source code, or system binaries, potentially leading to data exfiltration or further system compromise. Given the internet-facing nature of the host, successful exploitation could result in significant business impact including data breaches, regulatory fines, and reputational damage.

Exploitability rationale: The finding is rated High because the internet-facing Apache server running an unpatched version (2.4.49) is vulnerable to a critical path traversal flaw (CVE-2021-41773) that allows remote attackers to read arbitrary files, including sensitive system binaries, without requiring authentication or complex conditions.

Remediation: Elevate the security posture by upgrading the vulnerable Apache version and hardening configuration to prevent path traversal and remote code execution.

1. Upgrade Apache HTTP Server from version 2.4.49 to version 2.4.51 or later immediately.
2. Review and harden directory configurations to explicitly require 'Require all denied' for sensitive paths, avoiding reliance on default permissions.
3. Disable the `mod_cgi` module if CGI execution is not required for the application.
4. Analyze access logs for encoded path-traversal sequences (e.g., `'..%2f'`) to verify the system has not been compromised.

RAG references: CVE-2021-41773

#8 High Docker Engine API exposed without TLS on port 2375, internet-facing, allows container escape

Host: 192.168.1.150:2375

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: The Docker Engine API on host 192.168.1.150 is exposed unencrypted on port 2375, a critical misconfiguration that allows unauthenticated remote code execution. An attacker can leverage this exposure to execute arbitrary commands within the Docker host, effectively achieving a full container escape and gaining root-level control over the underlying operating system. This vulnerability enables severe business impacts including data exfiltration, lateral movement across the network, and complete compromise of the infrastructure.

Exploitability rationale: The combination of an internet-facing exposure, lack of authentication, and the ability to achieve full container escape and root control without TLS encryption results in a high-risk finding.

Remediation: Secure the Docker daemon by disabling the unencrypted remote API on port 2375 and enforcing TLS mutual authentication on port 2376 to prevent unauthenticated container escape.

1. Identify the Docker socket path and disable the unencrypted remote API by removing the bind address for port 2375 or setting the daemon's API endpoint to localhost only.
2. Configure the Docker daemon to listen on port 2376 with TLS enabled, ensuring mutual authentication is enforced by requiring client certificates.
3. Generate or obtain a valid client certificate and private key for the management host to authenticate against the Docker daemon.
4. Update the Docker daemon configuration (e.g., /etc/docker/daemon.json or systemd unit file) to enforce TLS on port 2376 and disable port 2375.
5. Restart the Docker service to apply the new configuration and verify that the API is no longer accessible without TLS.
6. Configure a firewall rule to allow traffic only from trusted management hosts to port 2376, explicitly blocking port 2375.
7. Verify the configuration by attempting to connect to the Docker API from an external host and confirming the connection is rejected or requires valid TLS credentials.
8. Audit container images and the host filesystem for any signs of persistence or unauthorized access resulting from the vulnerability.

RAG references: [service:docker](#)

#9 High PHP 8.0 running on Windows with CGI, internal-only (CVE-2024-4577)

Host: 192.168.1.200:8080

CVE: CVE-2024-4577

CVSS: 9.8

Risk score: 0.89 (asset criticality 0.5)

Context: This vulnerability affects PHP 8.0 running on Windows with CGI, allowing remote attackers to execute arbitrary code by sending a specially crafted HTTP request that triggers a buffer overflow in the PHP interpreter. An attacker could exploit this flaw to gain a foothold on the internal network, potentially leading to full system compromise, data exfiltration, or denial of service. Given the high CVSS score of 9.8, the business impact is severe, as it enables unauthorized access to critical internal systems and sensitive data without requiring user interaction or additional privileges.

Exploitability rationale: Despite the internal-only network exposure, the combination of a critical vulnerability (CVSS 9.8) with a public exploit and the ability to achieve remote code execution allows for immediate compromise without requiring complex attack conditions.

Remediation: Remediate the PHP CGI argument injection vulnerability by upgrading to a patched PHP version or applying the official workaround to block the injection character.

1. Upgrade the PHP installation to version 8.0.30 or higher, or migrate to a patched release of PHP 8.1, 8.2, or 8.3.
2. If an immediate upgrade is not possible, apply the official workaround to block the specific argument-injection character sequence.
3. Migrate from PHP-CGI to PHP-FPM where feasible, as PHP-FPM is the recommended architecture for Windows environments.
4. Restrict network access to the PHP endpoint and enable logging to monitor for the specific byte sequence used in the exploit.

RAG references: CVE-2024-4577

#10 High GitLab CE 16.1.0 self-hosted, account takeover possible via CVE-2023-7028, internal

Host: 192.168.1.130:8080

CVE: CVE-2023-7028

CVSS: 9.6

Risk score: 0.88 (asset criticality 0.5)

Context: The GitLab CE 16.1.0 instance on host 192.168.1.130 is vulnerable to CVE-2023-7028, a critical authentication bypass flaw allowing attackers to bypass the login page and access the application without valid credentials. An attacker could exploit this by sending a specific request to the login endpoint to gain unauthorized access to the GitLab instance, potentially leading to full account takeover and lateral movement within the internal network. This vulnerability poses a severe business impact by enabling complete compromise of the code repository, allowing data exfiltration, code injection, and unauthorized administrative control over the development environment.

Exploitability rationale: The vulnerability is rated High because it is an internet-facing service (exposed to the internet) with a critical authentication bypass flaw that allows unauthenticated access, and no patch is available for the affected version.

Remediation: Remediate the account takeover vulnerability by upgrading GitLab to a patched version, enforcing two-factor authentication, and auditing password reset logs to mitigate the risk of unauthenticated password resets.

1. Upgrade GitLab CE from version 16.1.0 to version 16.1.6 or any later patched release (e.g., 16.2.6, 16.3.4, 16.4.4, 16.5.6, 16.6.6, or 16.7.2).
2. Audit password reset logs for the affected accounts and force a password reset for any accounts that received a reset request.
3. Enable and enforce two-factor authentication (2FA) for all users, with mandatory enforcement for administrators.
4. Restrict access to the GitLab instance to authenticated internal networks where possible.

RAG references: CVE-2023-7028

#11 High Apache Tomcat 9.0.30 with manager app exposed and default credentials, internet-facing

Host: 192.168.1.30:8080

CVSS: 8.1

Risk score: 0.865 (asset criticality 0.8)

Context: The Apache Tomcat server on 192.168.1.30 is vulnerable because its management interface is publicly accessible with default credentials, allowing unauthorized users to gain full administrative control. Attackers can exploit this by authenticating as 'admin/admin' to deploy, remove, or modify web applications, effectively compromising the entire application server. This exposure leads to a critical business impact where attackers can execute arbitrary code, exfiltrate sensitive data, or pivot to other internal systems, potentially resulting in complete system compromise and data loss.

Exploitability rationale: The combination of an internet-facing exposure, default credentials, and the ability to deploy or modify applications allows attackers to easily achieve full system compromise without requiring complex exploits.

Remediation: The exposed Apache Tomcat manager interface with default credentials and internet access creates a critical attack surface that must be immediately restricted and hardened.

1. Restrict access to the Tomcat manager app (/manager) to trusted internal networks or a secure VPN by configuring firewall rules or network ACLs.
2. Replace the default manager credentials with strong, unique passwords and enable authentication mechanisms like Basic Auth or LDAP.
3. Disable or remove unnecessary manager features and ensure the application stack is updated to the latest supported version.
4. Place a reverse proxy or Web Application Firewall (WAF) in front of the Tomcat instance and enforce TLS (HTTPS) for all traffic.

RAG references: [service:http](#)

#12 High MongoDB 4.0 without authentication exposed on the internal network

Host: 192.168.1.120:27017

CVSS: 7.5

Risk score: 0.775 (asset criticality 0.5)

Context: The MongoDB instance on 192.168.1.120 is exposed without authentication, allowing any internal network user to connect and execute commands. Attackers can leverage this to perform unauthorized data exfiltration, modify or delete critical data, and potentially escalate privileges if the database contains weak credentials or writable configurations. This misconfiguration results in a high business impact by compromising data confidentiality, integrity, and availability, while potentially serving as a foothold for lateral movement within the internal network.

Exploitability rationale: The combination of an unauthenticated MongoDB instance on an internal network with an outdated version (4.0) that lacks known patches and has no authentication mechanism allows attackers to easily execute arbitrary commands and exfiltrate data.

Remediation: Enable MongoDB authentication and restrict network access to internal interfaces to prevent unauthorized data exposure.

1. Configure the MongoDB daemon to start with the '--auth' flag to enforce username/password authentication.
2. Create dedicated application users with the principle of least privilege, granting only necessary read/write permissions.
3. Bind the mongod service to the internal network interface (192.168.1.0/24) and disable listening on 0.0.0.0 or external interfaces.
4. Deploy firewall rules to allow connections only from specific application server IPs and deny all other traffic on port 27017.
5. Enable TLS encryption for all client connections and inter-node replication traffic to protect data in transit.

RAG references: service:mongodb

#13 Medium MySQL 5.7.27 with anonymous account and weak root password, internal-only

Host: 192.168.1.50:3306

CVSS: 7.5

Risk score: 0.725 (asset criticality 1.0)

Context: The internal MySQL server (192.168.1.50) is vulnerable due to the presence of an anonymous user and a weak root password, allowing attackers to gain immediate administrative access without authentication. In a real-world scenario, an attacker who gains network access to the internal segment could exploit these credentials to execute arbitrary SQL commands, exfiltrate sensitive database contents, or pivot to other systems. The business impact includes potential data breaches, loss of intellectual property, regulatory compliance violations, and significant reputational damage.

Exploitability rationale: The vulnerability is rated Medium because the service is internal-only, limiting exposure to attackers who must first gain network access to the internal segment, although the presence of an anonymous account and weak password significantly reduces the complexity of achieving administrative access once inside.

Remediation: Eliminate the anonymous account and enforce a strong root password to prevent unauthorized network access, then restrict the MySQL service to the internal interface and apply least-privilege principles.

1. Remove the anonymous user by executing the SQL command: `DELETE FROM mysql.user WHERE User='';`
2. Set a strong, complex root password using the `mysql_secure_installation` utility or the `ALTER USER` statement.
3. Bind the `mysqld` service to the internal network interface (192.168.1.0/24) and disable listening on 0.0.0.0.
4. Configure a firewall rule to allow inbound traffic only on port 3306 from the internal network range.
5. Create dedicated application users with minimal required privileges instead of using the root account for daily operations.
6. Enable TLS encryption for all database connections and rotate any potentially compromised credentials.

RAG references: service:mysql

#14 Medium Grafana 8.0.0 with arbitrary file read, internal-only (CVE-2021-43798)

Host: 192.168.1.110:3000

CVE: CVE-2021-43798

CVSS: 7.5

Risk score: 0.625 (asset criticality 0.5)

Context: The host 192.168.1.110 is running an outdated version of Grafana (8.0.0) which contains a critical vulnerability (CVE-2021-43798) allowing an authenticated user to read arbitrary files from the server's filesystem. An attacker who can gain access to the Grafana web interface could exploit this flaw to exfiltrate sensitive internal data, such as configuration files, logs, or database dumps, without needing elevated privileges. This unauthorized data access can lead to significant business impact, including intellectual property theft, compliance violations, and potential use of stolen credentials for further lateral movement within the network.

Exploitability rationale: The vulnerability is rated Medium because it requires authenticated access to an internal-only service, limiting the attack surface despite the critical severity of the arbitrary file read capability.

Remediation: Mitigate the arbitrary file read vulnerability by immediately upgrading Grafana to a patched version and enforcing network-level access controls to prevent unauthenticated exploitation.

1. Upgrade Grafana from version 8.0.0 to version 8.0.7 or a later patched release (e.g., 8.1.8, 8.2.7, or 8.3.1) to resolve the path traversal flaw.
2. Restrict access to the Grafana service on host 192.168.1.110 using a reverse proxy and firewall rules to allow only authenticated internal users.
3. Review application logs for suspicious path-traversal patterns in plugin URLs and investigate for potential data exfiltration.
4. Rotate any secrets or sensitive configuration keys stored in files that may have been exposed due to the vulnerability.

RAG references: CVE-2021-43798

#15 Medium WordPress 5.4 with multiple outdated plugins, internal-only, no known RCE CVE

Host: 192.168.1.160:80

CVSS: 6.5

Risk score: 0.575 (asset criticality 0.5)

Context: The host running WordPress 5.4 is vulnerable due to multiple outdated plugins, which often contain known security flaws that attackers can exploit to gain unauthorized access. Real-world attack scenarios include credential theft via SQL injection or cross-site scripting (XSS), allowing attackers to bypass authentication or inject malicious code into the application. The business impact involves potential data breaches, loss of customer trust, and regulatory fines, despite the current finding being internal-only with no known Remote Code Execution (RCE) CVE.

Exploitability rationale: The internal-only scope and lack of known RCE CVEs lower the risk, but the presence of multiple outdated plugins with known flaws and a moderate CVSS score indicates a plausible attack vector for unauthorized access or data theft.

Remediation: Mitigate the medium risk of WordPress 5.4 and outdated plugins by enforcing network segmentation, credential hardening, and patch management to close the attack surface.

1. Restrict access to the WordPress administrative interface (e.g., /wp-admin) to the internal network only using firewall rules or a reverse proxy.
2. Replace all default or weak credentials with strong, unique passwords and enable Multi-Factor Authentication (MFA) for all administrative accounts.
3. Update WordPress core to the latest stable version and replace all outdated plugins with their latest secure versions or remove unused components.
4. Deploy a Web Application Firewall (WAF) in front of the service to enforce TLS encryption and filter malicious traffic.

RAG references: [service:http](#)

#16 Low PostgreSQL 15.3, patched, listening only on localhost

Host: 192.168.1.170:5432

Risk score: 0.38 (asset criticality 0.5)

Context: The PostgreSQL service on host 192.168.1.170 is configured to listen only on localhost, which effectively restricts network exposure to the local machine. This configuration mitigates the primary attack vector of remote unauthorized access, as external attackers cannot initiate a connection. Consequently, the business impact is significantly reduced, as the database is not directly accessible from the public internet or untrusted networks.

Exploitability rationale: The service is patched and restricted to localhost, eliminating remote attack vectors and significantly reducing exploitability.

Remediation: The system is already configured with localhost-only binding, but further hardening requires enforcing strong authentication, minimal privileges, and encrypted communication to mitigate residual risks.

1. Verify that the PostgreSQL configuration file (postgresql.conf) explicitly sets `listen_addresses` to 'localhost' or '127.0.0.1' to prevent external network access.
2. Update the `pg_hba.conf` file to enforce SCRAM-SHA-256 authentication for all remote or local connections, replacing any older or insecure methods like MD5 or trust.
3. Review and restrict PostgreSQL roles to the principle of least privilege, ensuring users have only the specific permissions required for their functions.
4. Configure TLS/SSL certificates and update the `pg_hba.conf` to require SSL for all connections, ensuring no plaintext traffic is allowed.
5. Implement a firewall rule (e.g., iptables, nftables, or cloud security group) to explicitly allow traffic only from trusted IP ranges, even for localhost-bound services.

RAG references: `service:postgresql`

#17 Low Redis 7.2.3 with authentication enabled and ACL hardening, internal-only

Host: 192.168.1.180:6379

Risk score: 0.38 (asset criticality 0.5)

Context: The vulnerability finding indicates that a Redis instance on 192.168.1.180 is running version 7.2.3 with authentication enabled and ACL hardening applied, restricted to internal networks only. Since the service is explicitly configured for internal-only access and has implemented authentication and access control lists, there are no known public exploits or unauthenticated remote attack vectors targeting this specific configuration. Consequently, the business impact is minimal to non-existent, as the combination of network segmentation, authentication, and ACLs effectively mitigates the typical risks associated with exposed Redis instances.

Exploitability rationale: The service is rated Low because it is internal-only with authentication and ACL hardening applied, and there are no known public exploits for the specific version.

Remediation: The host is already hardened with authentication, ACLs, and internal binding; remediation focuses on maintaining the patched state and explicitly disabling dangerous commands to close remaining attack vectors.

1. Verify Redis is running the latest patched version (7.2.3 or newer) to address any known security updates.
2. Confirm the server binds strictly to localhost or internal network interfaces to prevent external access.
3. Review and enforce strong ACL user configurations to ensure only authorized commands are permitted.
4. Explicitly disable or rename dangerous commands such as CONFIG, FLUSHALL, and EVAL to prevent privilege escalation or data loss.
5. Enable TLS encryption for any network traffic to Redis to protect data in transit.

RAG references: service:redis

#18 Low OpenSSH 9.6p1 fully patched on a bastion host, internet-facing

Host: 192.168.1.190:22

Risk score: 0.38 (asset criticality 0.5)

Context: The finding indicates that the SSH service on the bastion host is fully patched, meaning the specific vulnerability associated with the finding does not exist. Since the system is patched and the CVE is listed as 'none', there is no active exploitability or immediate threat from known vulnerabilities. Consequently, the business impact is negligible, and no immediate remediation is required beyond maintaining the current patched state.

Exploitability rationale: The service is fully patched with no known CVEs and resides on a bastion host, significantly limiting attack complexity and business impact despite its internet-facing exposure.

Remediation: Although the OpenSSH service is patched, its internet-facing exposure requires additional hardening to mitigate brute-force attacks and unauthorized root access.

1. Verify the OpenSSH version is up to date and apply any latest upstream security patches.
2. Configure the sshd_config to disable root login by setting 'PermitRootLogin no'.
3. Enforce key-based authentication and disable password authentication by setting 'PasswordAuthentication no'.
4. Restrict SSH access via firewall rules to allow connections only from trusted management networks.
5. Enable verbose logging and configure monitoring tools to detect and alert on brute-force attempts.

RAG references: service:ssh

#19 Low nginx 1.25.3, no known vulnerabilities, internet-facing

Host: 192.168.1.40:80

Risk score: 0.34 (asset criticality 0.3)

Context: The finding indicates an internet-facing nginx 1.25.3 instance with no known vulnerabilities, meaning there are no publicly disclosed exploits targeting this specific version. While the lack of known CVEs suggests a lower immediate risk of remote code execution, the public exposure of a web server remains a significant attack surface for reconnaissance, denial of service via resource exhaustion, or potential exploitation of unpatched zero-day vulnerabilities if they are discovered. The primary business impact lies in the potential for service disruption or data exposure if an attacker successfully compromises the server, even without a specific CVE in the current database.

Exploitability rationale: The rating is Low because the service runs a patched version of nginx with no known vulnerabilities and lacks public exploits, despite the inherent risk of internet exposure.

Remediation: Although the nginx instance reports no known vulnerabilities, its internet-facing exposure and outdated version (1.25.3) necessitate hardening to mitigate potential misconfigurations and future threats.

1. Restrict access to the nginx service by placing it behind a reverse proxy or Web Application Firewall (WAF) and enforcing TLS encryption for all traffic.
2. Remove or disable administrative interfaces (e.g., /admin, manager consoles) and ensure they are not reachable from the public internet.
3. Update nginx from version 1.25.3 to the latest stable release to patch any unlisted security flaws and improve hardening options.
4. Ensure the server configuration does not expose default credentials and that any management access is limited to trusted internal networks or VPNs.

RAG references: service:http

#20 Low OpenSSH 8.2p1 on internal host, patched against known CVEs

Host: 192.168.1.20:22

CVE: CVE-2024-6387

CVSS: 3.1

Risk score: 0.285 (asset criticality 0.5)

Context: The vulnerability CVE-2024-6387 affects OpenSSH versions prior to 9.6p1, including the reported 8.2p1, by allowing an attacker to bypass the SSH protocol version check. This enables an adversary to downgrade the connection to an older, insecure protocol version, potentially bypassing security controls and facilitating man-in-the-middle attacks or exploitation of other known vulnerabilities in older SSH implementations. While the host is noted as patched against known CVEs, this specific finding indicates a version mismatch or incomplete patching, which could lead to unauthorized access or data interception if the server is exposed to untrusted networks.

Exploitability rationale: The vulnerability is mitigated by the patched version and the internal network exposure limits the attack surface, requiring active network access to exploit.

Remediation: The host is vulnerable to an unauthenticated remote code execution race condition in OpenSSH due to a version mismatch with the known fix, requiring an immediate upgrade to a patched release.

1. Verify the current OpenSSH version on host 192.168.1.20 is indeed 8.2p1.
2. Upgrade OpenSSH to version 9.8p1 or later to patch the signal-handler race condition.
3. If an immediate upgrade is not possible, temporarily set 'LoginGraceTime 0' in sshd_config to disable the grace-period race.
4. Restrict sshd access via firewall rules to only trusted management networks.
5. Ensure 'PermitRootLogin' is set to 'no' and enforce key-based authentication.

RAG references: CVE-2024-6387

Ranked Summary

Rank	Score	Exploitability	Finding	Host
1	0.99	High	Open Redis 3.2.0 without authentication, internet-facing	192.168.1.10
2	0.9	High	Apache Log4j 2.14.1 detected in a Java web app, internet-facing (CVE-2021-44228)	192.168.1.60
3	0.9	High	Apache Struts 2.5.16 running a legacy action, internet-facing (CVE-2017-5638)	192.168.1.100
4	0.89	High	Jenkins 2.426 running with CLI exposed, internet-facing (CVE-2024-23897)	192.168.1.70
5	0.89	High	Atlassian Confluence 7.4.0 server exposed to the internet (CVE-2022-26134)	192.168.1.80

Rank	Score	Exploitability	Finding	Host
6	0.89	High	Elasticsearch 1.4.2 without authentication and scripting enabled, internal (CVE-2015-1427)	192.168.1.90
7	0.89	High	Apache HTTP Server 2.4.49 with path traversal, internet-facing (CVE-2021-41773)	192.168.1.140
8	0.89	High	Docker Engine API exposed without TLS on port 2375, internet-facing, allows container escape	192.168.1.150
9	0.89	High	PHP 8.0 running on Windows with CGI, internal-only (CVE-2024-4577)	192.168.1.200
10	0.88	High	GitLab CE 16.1.0 self-hosted, account takeover possible via CVE-2023-7028, internal	192.168.1.130
11	0.865	High	Apache Tomcat 9.0.30 with manager app exposed and default credentials, internet-facing	192.168.1.30
12	0.775	High	MongoDB 4.0 without authentication exposed on the internal network	192.168.1.120
13	0.725	Medium	MySQL 5.7.27 with anonymous account and weak root password, internal-only	192.168.1.50
14	0.625	Medium	Grafana 8.0.0 with arbitrary file read, internal-only (CVE-2021-43798)	192.168.1.110
15	0.575	Medium	WordPress 5.4 with multiple outdated plugins, internal-only, no known RCE CVE	192.168.1.160
16	0.38	Low	PostgreSQL 15.3, patched, listening only on localhost	192.168.1.170
17	0.38	Low	Redis 7.2.3 with authentication enabled and ACL hardening, internal-only	192.168.1.180
18	0.38	Low	OpenSSH 9.6p1 fully patched on a bastion host, internet-facing	192.168.1.190
19	0.34	Low	nginx 1.25.3, no known vulnerabilities, internet-facing	192.168.1.40
20	0.285	Low	OpenSSH 8.2p1 on internal host, patched against known CVEs	192.168.1.20

Risk score = (CVSS/10 × 0.5) + (Exploitability × 0.3) + (Asset × 0.2). Exploitability: High=1.0, Medium=0.5, Low=0.1.